

LogPulse AI - Incident Report

Generated : 2026-05-29 16:42:44 UTC

Trace ID : d03268b4-d3c3-45dd-999e-ddcf44b56190

1 - Incident Summary

Incident ID : d03268b4
Category : Unknown
Severity : SeverityLevel.INFO
Source : Fallback
Cache : MISS

2 - Raw Log

16:42:11 FATAL OOMKilled: pod/worker-api exceeded memory limit 256Mi restarts=14

3 - Classification

Category : Unknown
Severity : SeverityLevel.INFO
Summary : Fallback classification triggered due to inference failure.
Source : Fallback

4 - Diagnosis

Root Cause : The pod/worker-api is repeatedly OOMKilled because its configured memory limit of 256Mi is insufficient for the application's runtime memory requirements, leading to 14 restarts.
Confidence : 95%
Reasoning : The raw log explicitly states 'FATAL OOMKilled: pod/worker-api exceeded memory limit 256Mi restarts=14'. The retrieved playbook, with high confidence, is titled 'Pod OOMKilled: Insufficient Memory Limit' and suggests increasing memory limits.
Source Tier : llm
Used History : No
Used Playbook : Yes

5 - Proposed Solution

Explanation:

Gemini unavailable; remediation derived from playbook 'Pod OOMKilled: Insufficient Memory Limit' (RAG confidence=0.7893). Review the playbook steps and adapt commands to the live environment before execution.

Steps:

- 1. Verify the `OOMKilled` status using `kubectl get pod <pod-name> -o yaml`.
- 2. Check container memory usage metrics (e.g., Prometheus/Grafana) to identify peak memory consumption.
- 3. Analyze application logs for clues about memory spikes or growth patterns.
- 4. Increase the `resources.limits.memory` for the affected container in its Deployment/StatefulSet manifest.
- 5. Roll out the updated deployment using `kubectl apply -f <manifest-file>` or `kubectl rollout restart deployment/<deployment-name>`.
- 6. name: my-app

Commands:

```
$ kubectl get pod <pod-name> -o yaml
$ kubectl apply -f <manifest-file>
$ kubectl rollout restart deployment/<deployment-name>
```

6 - Risk Assessment

Risk Level : **RISKLEVEL.WARNING**

Safe Commands (2):

- [OK] `kubectl get pod <pod-name> -o yaml`
- [OK] `kubectl apply -f <manifest-file>`

Flagged Commands (1):

- [!] `kubectl rollout restart deployment/<deployment-name>` [`kubectl\s+rollout\s+restart`]

7 - Operator Decision

Decision : OPERATORDECISION.APPROVED
Executed At : 2026-05-29T13:42:44.222271+00:00

Sandbox Results:

Command : `kubectl get pod <pod-name> -o yaml`
Exit : 0
Time : 0ms

Output:

```
[auto-approve] simulated: kubectl get pod <pod-name> -o yaml
```

Command : `kubectl apply -f <manifest-file>`
Exit : 0
Time : 0ms

Output:

```
[auto-approve] simulated: kubectl apply -f <manifest-file>
```

Command : `kubectl rollout restart deployment/<deployment-name>`
Exit : 0
Time : 0ms

Output:

```
[auto-approve] simulated: kubectl rollout restart deployment/<deployment-name>
```

8 - Historical Context

Times Seen : 1
First occurrence -- no prior incidents.

9 - Incident Timeline

[13:42:44] WORKFLOW_COMPLETED -> Completed in 14671ms